

False Positive Documentation

Kindly provide the answers for the following queries regarding the vulnerability.

1. Name of the Application

Flowdometer

2. Package ID/Version ID/Listing ID

a0N3A00000GB0VkUAL

3. Partner Name

Muse Operations

4. Document Filing Date (YYYY-MM-DD)

2025-11-23

5. Target URL (Optional)

--

6. Review Date (In Case if app already undergone security review)

--

Repeat Step 6 to 9 in case of multiple Vulnerability

7. Vulnerability Name

Sharing Violation – ListenerMasterConfigurationController.LookupFieldCreator

Meant for the Internal Salesforce Security Team

Final Comment from Salesforce Team	
Date of Review (YYYY-MM-DD)	
Result	

8. Detected By

Checkmarx Query 1031 (Sharing) – Scan ID 1009119

9. Detailed explanation for considering False Positive (Elaborate as much as possible)

LookupFieldCreator is a Queueable helper that provisions metadata lookups after a Listener is created. It never queries or mutates subscriber business data; it only calls Metadata API and, if needed, writes an error string back to the originating Listener record (behind FLS guards). The class now declares **public inherited sharing**, which is Salesforce's recommended pattern for queueables because it automatically honors the sharing context of the user who enqueued the job. The Checkmarx Sharing rule only understands **with sharing**, so it continues to flag the class even though sharing is enforced via inheritance. All entry points (e.g., **ListenerMasterConfigurationController**) are **with sharing**, so the queueable inherits their context. No sensitive data access occurs, and the Metadata API already enforces its own permissions. Therefore this finding is a tooling limitation rather than a real exposure.

10. Evidence (screenshots of the code, implementation, Command Output)

Source file:

force-app/main/default/classes/controllers/ListenerMasterConfigurationController.cls

```
```apex
```

```
// Queueable to perform Metadata callout after DML is committed
```

```
public inherited sharing class LookupFieldCreator implements Queueable,
Database.AllowsCallouts {
```

```
String targetObject;
```

```
String fieldLabel;
```

```
String referenceTo;
```

```
Id listenerId;
```

```
public LookupFieldCreator(String tgt, String lbl, String ref, Id lst) { ... }
```

```
public void execute(QueueableContext qc) {
```

```
// Calls MetaDataUtilityCls.createLookupField and only writes to Listener__c
```

```
// All writes guarded by Schema.sObjectType.Listener__c FLS checks
```

```
}
```

```
}
```

```
```
```

Checkmarx Report: `checkmarx5\_report\_phxcmarxwp001\_9116.xml / ChecCheckmarx\_Nov\_2025\_v2\_71\_Scan\_Results.pdf`, Query 1031 flagging line 469 even though the class is `inherited sharing`.

References

1. *Name of the Application*

Hello for Salesforce

2. *Package ID/Version ID/Listing ID*

a0N4V00000HwgXYZK

3. *Partner Name*

Hari Krishna Company Ltd

4. *Document Filing Date (YYYY-MM-DD)*

2022-08-20

5. *Target URL (Optional)*

<https://test.world.com/>

6. *Review Date (In Case if app already undergone security review)*

2022-08-15

Repeat Step 7 to 10 in case of multiple Vulnerability

7. *Vulnerability Name*

CRUD Violation

Meant for the Internal Salesforce Security Team

| | |
|---|--|
| <i>Final Comment from Salesforce Team</i> | |
| <i>Date of Review (YYYY-MM-DD)</i> | |
| <i>Result</i> | |

8. *Detected By*

Checkmarx PMD ~~Security Review~~ Others, If case of others Please provide details :

9. *Detailed explanation for considering False Positive (Elaborate as much as possible)*

CRUD Violation Issue was flagged during security review as it was mentioned under explanation that no CRUD/FLS check was performed before the DML operation. But it is a false positive as a relevant check is performed in the BaseHelper class and the 'checkCRUDPermissionOnSubject' object is being called before the DML operation. 'checkCRUDPermissionOnSubject' object necessary to check the CRUD permission along with the FLS check. Relevant evidence provided in the next section. Hence the following vulnerability should be marked as False positive and marked closed.

10. Evidence (screenshots of the code, implementation, Command Output)

```
Dummy.cls public static Dummy (){
BaseHelper.checkCRUDPermissionOnSubject('Test', 'updateable');      Test
Item = new Test();      Item.Id = accordionItmId;      Item.Dummy = User.Id;
update Item;  } BaseHelper.cls public static void
checkCRUDPermissionOnSubject(String subjName, Set<String>
permissionTypes){  PermissionWrapper permissionWrapperObj = new
PermissionWrapper();  permissionWrapperObj =
setPermissionWrapper(subjName, null);  Schema.DescribeObjectResult
subjResult = permissionWrapperObj.subjResult;  Boolean
isPermissionTypeIssue = isCRUDPermissionTypeIssueOnSubject(subjName,
permissionTypes);  if (isPermissionTypeIssue){  throw new
DataTableHelper.OtherException('Insufficient \'' + permissionTypes + '\'
permission access to subject \'' + subjResult.getLabel() + '\');  } }
```

Repeat Step 11 to 14 in case of multiple Vulnerability

11. Vulnerability Name

Sharing Violation

Meant for the Internal Salesforce Security Team

| | |
|---|--|
| Final Comment from Salesforce Team | |
| Date of Review (YYYY-MM-DD) | |

| | |
|---|--|
| <i>Final Comment from Salesforce Team</i> | |
| <i>Result</i> | |

12. *Detected By*

Chimera PMD Security Review Others, If case of others Please provide details :

13. *Detailed explanation for considering False Positive (Elaborate as much as possible)*

Sharing Violation issue was flagged during the chimera scan as class was declared as without sharing. As this class has to be called by all the users whoever wants to access the application (even outside the org also) for the installation. Sharing permission is explicitly omitted and hence it is declared as Without Sharing. As this is a business requirement this issue should be marked as false positive and closed.

14. *Evidence (screenshots of the code, implementation, Command Output)*

```
global without sharing class PostInstallClass implements InstallHandler {
global void onInstall(InstallContext context) {
SubjectIconController.createSubjectIconMetadata();
if(context.previousVersion() == null) {
DataFactory.createDataFactorySetting();    }
}
```

